# 01. 회귀 소개

- 데이터 값이 평균과 같은 일정한 값으로 돌아가려는 경향을 이용한 통계학 기법
- 여러 개의 독립변수와 한 개의 종속변수 간의 상관관계를 모델링하는 기법을 통칭

| 독립변수 개수 | 회귀계수의 결합 |
|---|---|
| 1개: 단순회귀 | 선형: 선형 회귀 |
| 여러 개: 다중 회귀 | 비선형: 비선형 회귀 |

- 머신러닝 관점에서의 회귀
  - **독립변수는 피처, 종속변수는 결정값**
  - 핵심: 주어진 피처와 결정 값 데이터 기반에서 학습을 통해 최적의 회귀계수를 찾아내는 것.
- 지도학습은 분류, 회귀 두가지로 나뉨.
  - 분류: 예측값이 카테고리와 같은 이산형 클래스값
  - 회귀: 예측값이 연속형 숫자값

## 선형 회귀

- 실제 값과 예측값의 차이를 최소화하는 직선형 회귀선을 최적화하는 방식으로, 머신러닝에서 가장 많이 사용되는 회귀.
- **일반 선형 회귀**는 규제를 적용하지 않은 모델.
  - 규제 : 일반 선형 회귀의 과적합 문제를 해결하기 위해, 회귀 계쑤에 페널티 값을 적용하는 것.
- 규제 방법에 따라 다시 여러 유형으로 나뉨 → **릿지, 라쏘, 엘라스틱넷, 로지스틱 회귀**

# 02. 단순 선형 회귀를 통한 회귀 이해

- 단순 선형 회귀: 독립변수, 종속변수 각각 하나인 선형 회귀

## 잔차(Residual)

- (실제 값 - 회귀모델 값) 차이에 따른 오류값
- 잔차는 +, -가 될 수 있으므로 , 잔차 합을 계산할 때는 아래 2가지 방법 사용.
    - 1. 절댓값을 취하여 더하기 (Mean Absolute Error)
    - **2. 제곱합 (RSS, Residual Sum of Square)** → 일반적으로 사용하는 방식.
        - 독립변수, 종속변수는 RSS에서 상수.
        - RSS= 비용(Cost)이며,  w변수(회귀계수)로 구성되는 RSS는 비용함수.

$$RSS(w_0, w_1) = \frac{1}{N}\sum_{i=1}^{N}(y_i - (w_0 + w_1 * x_i))^2$$

**($i$는 1부터 학습 데이터의 총 건수 $N$까지)**

- 머신러닝 기반 회귀에서는 RSS 를 최소로 하는 회귀계수 w0, w1을 학습을 통해 찾는 것이 핵심.
    - **비용 함수(또는 손실 함수, loss function)가 반환하는 값을 지속해서 감소시키고 최종적으로 더 이상 감소하지 않는 최소의 오류 값을 구한다.**

# 03. 비용 최소화하기 - 경사하강법(Gradient Descent) 소개

## 경사하강법

- 반복적인 계산을 통해 점진적으로 W 파라미터 값을 업데이트하면서 오류 값이 최소가 되는 W 파라미터를 구하는 방식.
- 비용함수의 반환 값(예측값-실제값)이 작아지는 방향성으로 W 파라미터를 지속적으로 보정. 더 이상 그 값이 작아지지 않으면, 그 오류 값을 최소 비용으로 판단하고 그때의 W 값= 최적의 파라미터
- **비용함수 RSS(w0, w1)= R(w)를 w0, w1에 대해 편미분 결괏값** 구함.
- **새로운 w1= 이전 w1 - [ w1로 편미분 결괏값 *보정계수(학습률) ] 식으로 w를 업데이트.**

$$w_1 + \eta\frac{2}{N}\sum_{i=1}^{N} x_i * (\text{실제값}_i - \text{예측값}_i),\ w_0\text{을}\ w_0 + \eta\frac{2}{N}\sum_{i=1}^{N}(\text{실제값}_i - \text{예측값}_i)$$

- Step 1: $w_1, w_0$를 임의의 값으로 설정하고 첫 비용 함수의 값을 계산합니다.
- Step 2: $w_1$을 $w_1 + \eta \frac{2}{N} \sum_{i=1}^{N} x_i * (\text{실제값}_i - \text{예측값}_i)$, $w_0$을 $w_0 + \eta \frac{2}{N} \sum_{i=1}^{N} (\text{실제값}_i - \text{예측값}_i)$으로 업데이트한 후 다시 비용 함수의 값을 계산합니다.
- Step 3: 비용 함수가 감소하는 방향성으로 주어진 횟수만큼 Step 2를 반복하면서 $w_1$과 $w_0$를 계속 업데이트합니다.

### 함수 구현

- get_weight_updates 함수: 편미분 값 * 보정계수 반환
- gradient_descent_steps 함수: w0, w1의 업데이트 된 값(new w0, new w1)을 반환.
    - w0, w1 모두 1x1 배열 형태인 0으로 초기화. (간단한 값 아무거나 출발점으로 지정 가능) → 경사하강법에서 w1, w0이 아무값이어도, 적절한 학습률을 사용하면 최적의 w0, w1을 점점 찾아가기 때문.
    - iters만큼 반복적으로 get_weight_updates를 호출해 w0, w1 편미분 수행 후, 새로운 w0와 w1을 계산함
- get_cost 함수: 예측값과 실제값의 차이인 RSS를 계산하는 비용함수 생성 예측 오류 계산

## 확률적 경사하강법

- 경사 하강법은 모든 학습 데이터에 대해 반복적으로 비용함수 최소화를 위한 값을 업데이트 → 수행시간 ↑
- **일부 데이터만 이용해 w가 업데이트되는 값 계산 → 속도 빠름**
- **대용량 데이터의 경우, 다음 방법 이용**
    - **확률적 경사 하강법**
    - **미니 배치 확률적 경사 하강법**

### 함수 구현

- stochastic_gradient_descent_steps 함수: gradient_descent_steps 함수와 비슷하나, X,y 데이터에서 랜덤하게 batch_size 만큼 추출한 sample_X, sample_y 이용.

# 04. 사이킷런 LinearRegression을 이용한 보스턴 주택 가격 예측

## LinearRegression 클래스 - Ordinary Least Squares

- RSS를 최소화해 OLS(Ordinary Least Squares) 추정 방식으로 구현한 클래스
- **fit() 메서드로 X,y 입력 → 회귀계수인 W를 coef_속성에 저장**

| | | |
|---|---|---|
| 입력 파라미터 | **fit_intercept** | 디폴트 True.<br>intercept 값을 계산할지말지 지정. False 이면 intercept=0 |
| | **normalize** | 디폴트 False.<br>False 이면 이 파라미터가 무시됨. True이면 회귀 수행전 입력데이터 세트를 정규화함. |
| 속성 | **.coef_** | fit() 메서드를 수행했을 때, 회귀계수가 배열 형태로 저장하는 속성. |
| | **.intercept_** | intercept 값 |

- OLS의 다중공산성 문제: 입력피처 간 상관관계가 높은 경우, 분산이 매우 커짐.
    - 해결: 독립적인 중요한 피처만 남기고 규제 적용, PCA를 통해 차원 축소.

## 회귀 평가 지표

**일반적으로 회귀의 성능을 평가하는 지표**

| 평가 지표 | 설명 | 수식 |
|---|---|---|
| MAE | Mean Absolute Error(MAE)이며 실제 값과 예측값의 차이를 절댓값으로 변환해 평균한 것입니다. | $MAE = \frac{1}{n}\sum_{i=1}^{n}\lvert Yi - \hat{Y}i\rvert$ |
| MSE | Mean Squared Error(MSE)이며 실제 값과 예측값의 차이를 제곱해 평균한 것입니다. | $MSE = \frac{1}{n}\sum_{i=1}^{n}(Yi - \hat{Y}i)^2$ |
| RMSE | MSE 값은 오류의 제곱을 구하므로 실제 오류 평균보다 더 커지는 특성이 있으므로 MSE에 루트를 씌운 것이 RMSE(Root Mean Squared Error)입니다. | $RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(Yi - \hat{Y}i)^2}$ |
| $R^2$ | 분산 기반으로 예측 성능을 평가합니다. 실제 값의 분산 대비 예측값의 분산 비율을 지표로 하며, 1에 가까울수록 예측 정확도가 높습니다. | $R^2 = \frac{\text{예측값 } Variance}{\text{실제값 } Variance}$ |

**각 평가 방법에 대한 사이킷런의 API &**
**cross_val_score나 GridSearchCV에서 평가 시 사용되는 scoring 파라미터의 적용 값**

| 평가 방법 | 사이킷런 평가 지표 API | Scoring 함수 적용 값 |
|---|---|---|
| MAE | metrics.mean_absolute_error | 'neg_mean_absolute_error' |
| MSE | metrics.mean_squared_error | 'neg_mean_squared_error' |
| RMSE | metrics.mean_squared_error를 그대로 사용하되 squared 파라미터를 False로 설정. | 'neg_root_mean_squared_error' |
| MSLE | metrics.mean_squared_log_error | 'neg_mean_squared_log_error' |
| $R^2$ | metrics.r2_score | 'r2' |

- RMSE는 따로 API를 제공하지 않음. np.sqrt( mean_squared_error(y_test, y_preds) ) 식으로 MSE에 루트를 씌워 이용한다.

## 주의할 점

**: Scoring 파라미터값 ex. neg_mean_absolute_error =  -1*metrics.mean_absolute_error.**
사이킷런 평가 지표 API인 metrics.mean_absolute_error은 정상적으로 양수 값을 반환함.

- **neg_ 접두사 : Negative(-)값을 가짐.**
- MAE, MSE, RMSEm MSLE는 음수 불가능. 작을수록 좋은 평가 결과라는 의미를 가짐.
- 그러나 Scoring 함수는 클수록 좋은 평가 결과로 자동인식함.

→ ( -1*원래 평가지표값 ) 보정을 하여, 작은 값이 더 큰 숫자로 인식되게 하는 것.

# LinearRegression을 이용해 보스턴 주택 가격 회귀 구현



- CRIM: 지역별 범죄 발생률
- ZN: 25,000평방피트를 초과하는 거주 지역의 비율
- INDUS: 비상업 지역 넓이 비율
- CHAS: 찰스강에 대한 더미 변수(강의 경계에 위치한 경우는 1, 아니면 0)
- NOX: 일산화질소 농도
- RM: 거주할 수 있는 방 개수
- AGE: 1940년 이전에 건축된 소유 주택의 비율
- DIS: 5개 주요 고용센터까지의 가중 거리
- RAD: 고속도로 접근 용이도
- TAX: 10,000달러당 재산세율
- PTRATIO: 지역의 교사와 학생 수 비율
- B: 지역의 흑인 거주 비율
- LSTAT: 하위 계층의 비율
- MEDV: 본인 소유의 주택 가격(중앙값)

클래스를 이용하기 전, 간단하게 회귀직선과 산점도를 그려 관계를 확인한다.

## 회귀 직선과 산점도 여러 개 그리기 - seaborn의 replot(), matplotlib의 subplots()

- seaborn의 replot(): X, Y 산점도와 함께 회귀직선을 그려줌
- matplotlib의 subplots(): 여러 그래프를 한번에 표현할 때 사용. ncols=열방향으로 위치할 그래프수. nrows= 행방향으로 위치할 그래프수

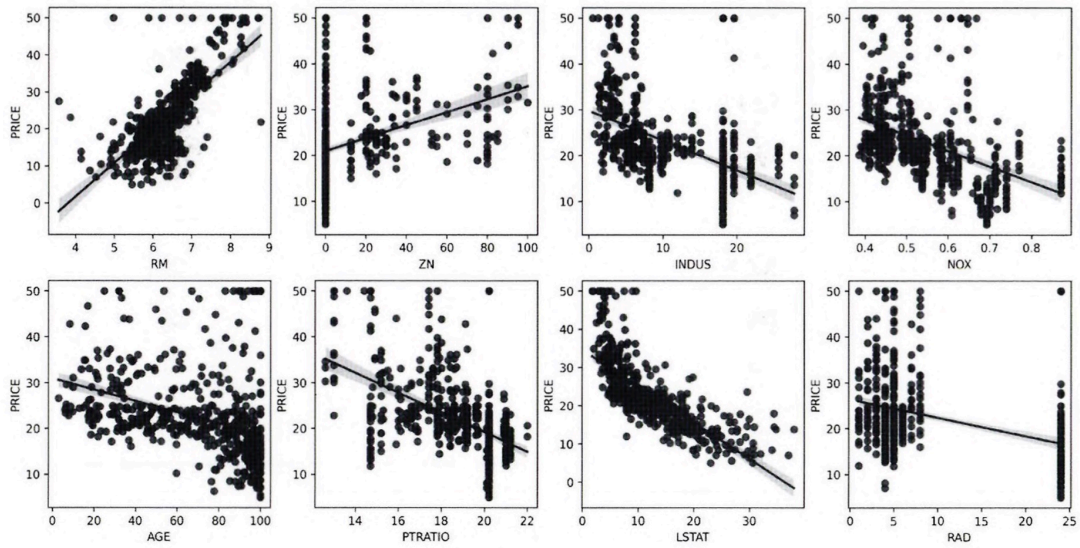
```
#총 8개 칼럼에 대해 값이 증가할수록 PRICE 칼럼이 어떻게 변하는지 확인.

#2개의 행, 4개의열을 가진 subplots를 이용. axs는 4x2개의 ax를 가짐.
fig, axs= plt.subplots(figsize=(16,8), ncols=4, nrows=2)  #2*4 짜리 그래프 공간 만들기

#axs는 이런 형태임 ex. axs[0][0], axs[0][1], axs[0][2], axs[0][3] ..
lm_features=  ['RM', 'ZN', 'INDUS', 'NOX', 'AGE', 'PTRATIO', 'LSTAT', 'RAD'] #X변수 8개

for i, feature in enumerate(lm_features):
    row=int(i/4) #처음 4개는 row=0, 나중 4개는 row=1
    col=i%4 #0, 1, 2, 3, 0, 1, 2, 3 열.

    #시본의 regplot을 이용해 산점도와 회귀직선을 함께 표현
    sns.regplot(x=feature, y='PRICE', data=bostonDF, ax=axs[row][col]) #x, y, data, ax(
```




4x2

# 05. 다항 회귀와 과(대)적합/과소적합 이해

## 다항 회귀 이해

- 다항회귀도 선형회귀의 일종.
  - x에 대해서는 곡선 → 그래프는 곡선
  - 회귀계수에 대해서는 선형. → 회귀계수 b0, b1, b2이 제곱되거나 서로 곱해지지 않고 1차로 들어감
  - cf. 선형/ 비선형 회귀를 나누는 기준 : 회귀계수가 선형/비선형인지에 따름. (독립변수의 선형/비선형 여부와 무관)

- 다항회귀를 위한 사이킷런 클래스는 존재 x. → **비선형 함수를 선형 모델에 적용시키는 방법 사용**

### PolynomialFeatures 클래스

- degree 파라미터를 통해 입력받은 단항식 피처(X)를 degree에 해당하는 다항식 피처(X)로 변환 **(= 회귀모델에 입력으로 넣을 새로운 데이터(X))**
- 객체.**fit(), 객체.transform()** 메서드로 변환 작업. (or **fit_transform**). transform 반환값= 변환된 다항식 계수 피처
- 단항 계수 피처 -> 2차 다항 계수로 변환하는 경우

단항 계수 피처 $[x_1, x_2]$를 2차 다항 계수 $[1, x_1, x_2, x_1^2, x_1x_2, x_2^2]$로 변경

### 다항 회귀 구현

- **Polynomial Features 클래스를 이용해 n차에 맞게 변환한 새로운 피처(X) 만들기**
- **결정값(y, 정답) 구하기**
- **LinearRegression 모델 생성후, fit(X, y)**
- **모델.coef 로 계수 확인**

```
#정답인 y를 구하는 함수
def polynomial_func(X):
    y = 1 + 2*X[:,0] + 3*X[:,0]**2 + 4*X[:,1]**3
    return y

# Pipeline 객체로 Streamline하게 polynomial Feature 변환과 Linear Regression을 연결
model = Pipeline([('poly', PolynomialFeatures(degree=3)),
                  ('linear', LinearRegression())])

X = np.arange(4).reshape(2,2) #변환전 X. (x1, x2)가 두개(두개의 행)
y = polynomial_func(X)

model = model.fit(X, y)
#X_poly = PolynomialFeatures(degree=3).fit_transform(X)
#LinearRegression().fit(X_poly, y) 을 한다.

print('Polynomial 회귀 계수\n', np.round(model.named_steps['linear'].coef_, 2))
```

## 다항 회귀를 이용한 과소적합 및 과적합 이해

- 다항회귀의 차수(degree)를 높일수록, 학습 데이터에만 너무 맞춘 과적합이 이루어진다. → 테스트 데이터 환경에서는 예측 정확도가 떨어지는 문제 발생.
- 좋은 예측 모델은 학습 데이터의 패턴을 잘 반영하면서도 degree가 적당한, 균형 잡힌 모델

## 편향-분산 트레이드오프(Bias-Variance Trade off)

### 편향(Bias)

- 모델이 실제 관계를 너무 단순하게 가정해서 생기는 오차. 지나치게 한 방향성으로 치우친 경향.
- 매우 단순한 모델(Degree=1)은 고편향을 가짐.

### 분산(Variance)

- 학습 데이터가 조금만 달라져도 모델이 크게 달라지는 정도
- 복잡한 모델(Degree=15)은 고분산성을 가짐.

### Trade off

- 편향, 분산은 한쪽이 높으면 한쪽이 낮아지는 Trade-off 관계.
- **편향이 높으면, 분산이 낮아짐(과소적합)/ 분산이 높으면, 편향이 낮아짐(과적합)**
- 편향, 분산이 트레이드오프를 이루면서 **오류(cost) 값이 최대로 낮아지는 모델**이 가장 좋다.

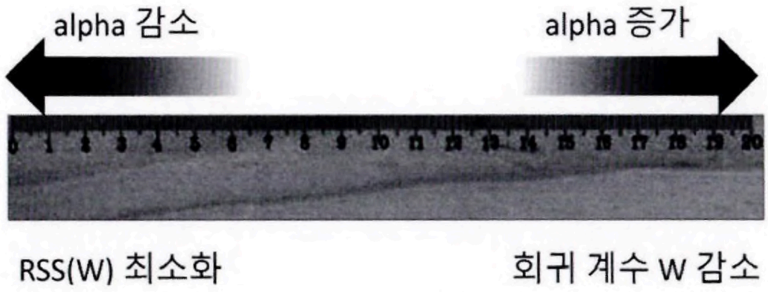
# 06. 규제 선형 모델 - 릿지, 라쏘, 엘라스틱넷

## 규제 선형 모델의 개요

- 비용함수가 RSS를 최소화하는 것만 고려하면, 학습 데이터에 지나치게 맞추게 되고 Degree가 커져 회귀계수(W)가 커지는 문제.
- → **RSS 최소화 방법** + **회귀계수 크기 제어** **균형 필요**

$$\text{비용 함수 목표} = \text{Min}(\text{RSS(W)} + \text{alpha} * \|W\|_2^2)$$

- alpha: 학습 데이터 적합 정도와 회귀 계수 값의 크기 제어를 수행하는 튜닝 파라미터
  - alpha 값을 크게 하면, 비용 함수는 회귀계수 W의 값을 작게 해 과적합을 개선
  - alpha 값을 작게 하면, W 값이 커져도 어느 정도 상쇄가 가능하므로 학습 데이터의 적합을 더 개선
  - → **alpha를 0에서부터 값을 증가시켜, 회귀 계수 값의 크기를 감소시킨다.**




〈alpha 튜닝 파라미터를 통한 RSS 최소화와 회귀 계수 크기 감소의 균형 조정〉

- **규제(Regularization) : 비용함수에 alpha 값으로 패널티를 부여해, 회귀계수값의 크기를 감소시켜 과적합을 개선하는 것**.
    - L2 규제 : 위와 같이 W제곱에 대해 패널티 부여 ≫ 릿지 회귀
    - L1 규제 : |W|에 패널티 부여 ≫ 라쏘 회귀

## 릿지 회귀

- 선형 회귀에 L2 규제를 추가한 회귀 모델
- L2 규제는 상대적으로 큰 회귀 계수값의 예측 영향도를 감소시키기 위해서 회귀 계수값을 더 작게 만드는 규제 모델
- **Ridge 클래스**를 통해 구현

## 라쏘 회귀

- 선형 회귀에 L1 규제를 적용한 모델
- L1 규제는 예측 영향력이 작은 피처의 회귀 계수를 0 으로 만들어 회귀 예측 시 피처가 선택되지 않게 함
- **Lasso 클래스**를 통해 구현

## 엘라스틱넷 회귀

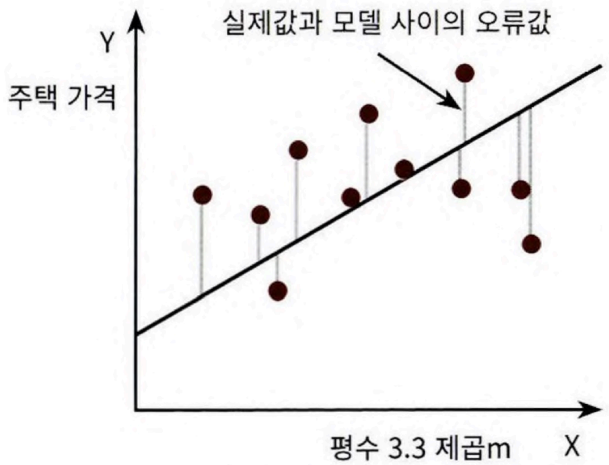
- L2, L1 규제를 함께 결합한 모델
- L1 규제로 피처의 개수를 줄임과 동시에 L2 규제로 계수 값의 크기를 조정함
- **ElasticNet 클래스**를 통해 구현

## 선형 회귀 모델을 위한 데이터 변환

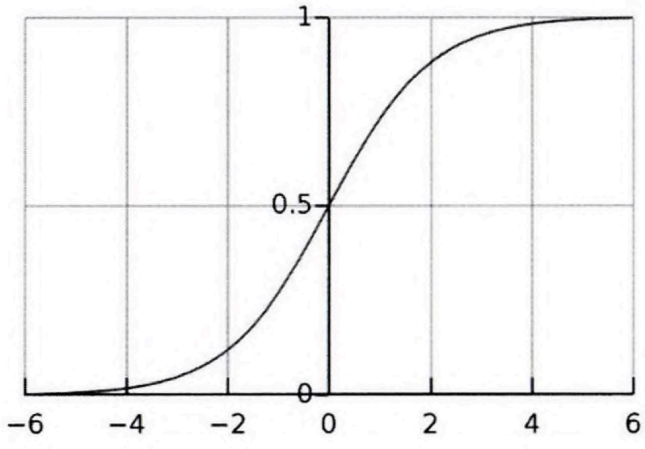
1. StandardScaler 클래스를 이용해 평균이 0, 분산이 1인 표준 정규 분포를 가진 데이터 세트로 변환하거나 MinMaxScaler 클래스를 이용해 최솟값이 0이고 최댓값이 1인 값으로 정규화를 수행합니다.
2. 스케일링/정규화를 수행한 데이터 세트에 다시 다항 특성을 적용하여 변환하는 방법입니다. 보통 1번 방법을 통해 예측 성능에 향상이 없을 경우 이와 같은 방법을 적용합니다.
3. 원래 값에 log 함수를 적용하면 보다 정규 분포에 가까운 형태로 값이 분포됩니다. 이러한 변환을 로그 변환(Log Transformation)이라고 부릅니다. 로그 변환은 매우 유용한 변환이며, 실제로 선형 회귀에서는 앞에서 소개한 1, 2번 방법보다 로그 변환이 훨씬 많이 사용되는 변환 방법입니다. 왜냐하면 1번 방법의 경우 예측 성능 향상을 크게 기대하기 어려운 경우가 많으며 2번 방법의 경우 피처의 개수가 매우 많을 경우에는 다항 변환으로 생성되는 피처의 개수가 기하급수로 늘어나서 과적합의 이슈가 발생할 수 있기 때문입니다.

# 07. 로지스틱 회귀

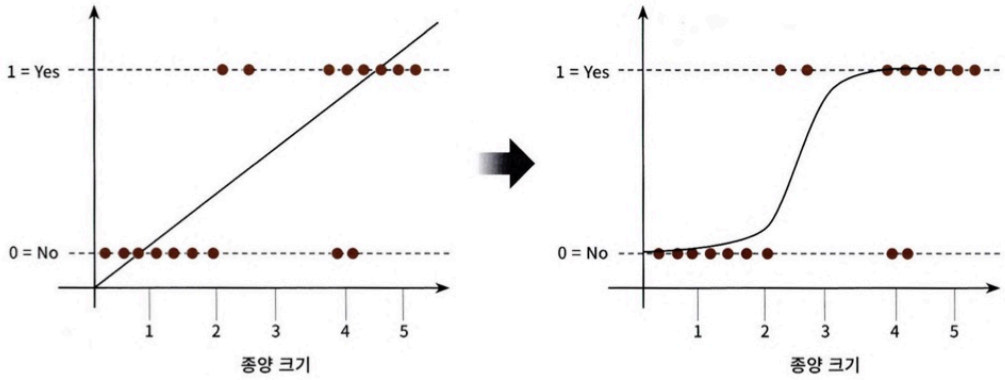
- 분류에 사용되는 선형 모델
- 일반적으로 이진 분류뿐 아니라 희소 영역의 분류 ( 텍스트 분류 ) 영역에서 뛰어난 예측 성능을 보임
- 선형 함수의 회귀 최적선을 찾는 것이 아니라 시그모이드 (Sigmoid) 함수 최적선을 찾고 이 시그모이드 함수의 반환값을 확률로 간주해 확률에 따라 분류 결정




〈 선형 회귀의 선형 함수 〉




〈 로지스틱 회귀의 시그모이드 함수 〉



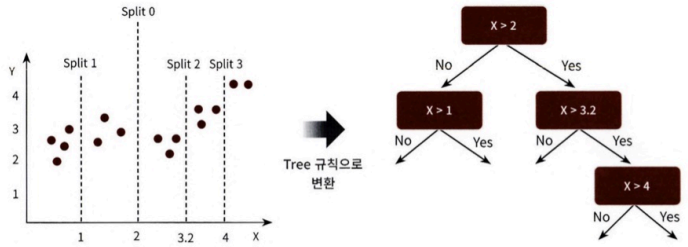

사이킷런은 로지스틱 회귀를 위해서 LogisticRegression 클래스를 제공합니다. LogisticRegression 클래스의 회귀 계수 최적화는 본 장의 초반부에 소개해 드린 경사 하강법 외에 다양한 최적화 방안을 선택할 수 있습니다. LogisticRegression 클래스에서 solver 파라미터의 'lbfgs', 'liblinear', 'newton-cg', 'sag', 'saga' 값을 적용해서 최적화를 선택할 수 있습니다.

- lbfgs: 사이킷런 버전 0.22부터 solver의 기본 설정값입니다. 메모리 공간을 절약할 수 있고, CPU 코어 수가 많다면 최적화를 병렬로 수행할 수 있습니다.
- liblinear: 사이킷런 버전 0.21까지에서 solver의 기본 설정값입니다. 다차원이고 작은 데이터 세트에서 효과적으로 동작하지만 국소 최적화(Local Minimum)에 이슈가 있고, 병렬로 최적화할 수 없습니다.
- newton-cg: 좀 더 정교한 최적화를 가능하게 하지만, 대용량의 데이터에서 속도가 많이 느려집니다.
- sag: Stochastic Average Gradient로서 경사 하강법 기반의 최적화를 적용합니다. 대용량의 데이터에서 빠르게 최적화합니다.
- saga: sag와 유사한 최적화 방식이며 L1 정규화를 가능하게 해줍니다.

# 08. 회귀트리

- 선형/비선형 회귀 함수를 이용하지 않고, 결정 트리와 같은 트리 기반의 알고리즘을 사용하여 회귀 예측을 수행

이 데이터 세트의 X 피처를 결정 트리 기반으로 분할하면 X값의 균일도를 반영한 지니 계수에 따라 다음 그림의 왼쪽과 같이 분할할 수 있습니다. 루트 노드를 Split 0 기준으로 분할하고 이렇게 분할된 규칙 노드에서 다시 Split 1과 Split 2 규칙 노드로 분할할 수 있습니다. 그리고 Split 2는 다시 재귀적으로 Split 3 규칙 노드로 다음 그림의 오른쪽과 같이 트리 규칙으로 변환될 수 있습니다.




리프 노드 생성 기준에 부합하는 트리 분할이 완료됐다면 리프 노드에 소속된 데이터 값의 평균값을 구해서 최종적으로 리프 노드에 결정 값으로 할당합니다.

